

Agent-Friendly范式探索

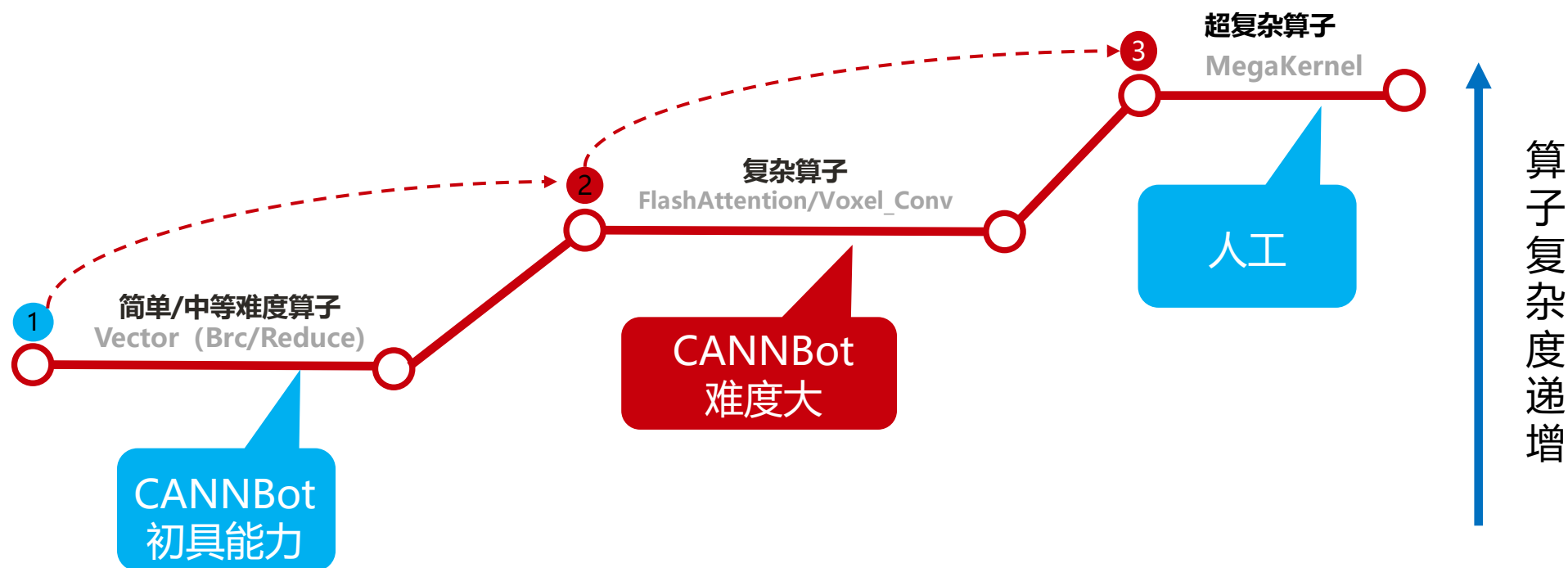
---基于CANNBot的高性能复杂算子生成

陈振宇(cannbot-dsl发起者)、樊玉伟(cann-bench committer)、
沈敏(cannbot maintainer)、杨继伟(cannbot-dsl committer)

<https://gitcode.com/cann/cannbot-dsl>

CANNBot生成算子的进展

基于CANNBot用Ascend C能够自动生成简单&中等难度的Vector/Matmul算子，且部分可以调优到比较好高性能。但对于复杂融合算子，受限于问题本质复杂度和LLM的能力，基于Ascend C的API还无法实现极致性能。



CANN-Bench打榜

CANN

CANNBot生成复杂算子遇到的问题示例

1 困难的NBuffer的设计

不同的preload策略对UB的分配有影响，容易计算错误

2 API使用选择时间长

API的原子性不足，需要多个API配合完成

 设计阶段

3 数据加载环节错误率高

ND2NZ参数语义不够直观；GQA多组合并时间偏移计算复杂

4 复杂的同步指令排布

PingPong后代码膨胀，代码逻辑变复杂

 开发阶段

5 同步bug占比高且难定位

跨核同步流水链特别长，同步点距离很远

6 性能调优代码改动大

每次调优都是代码的大规模改动，很难稳定下来，效率低

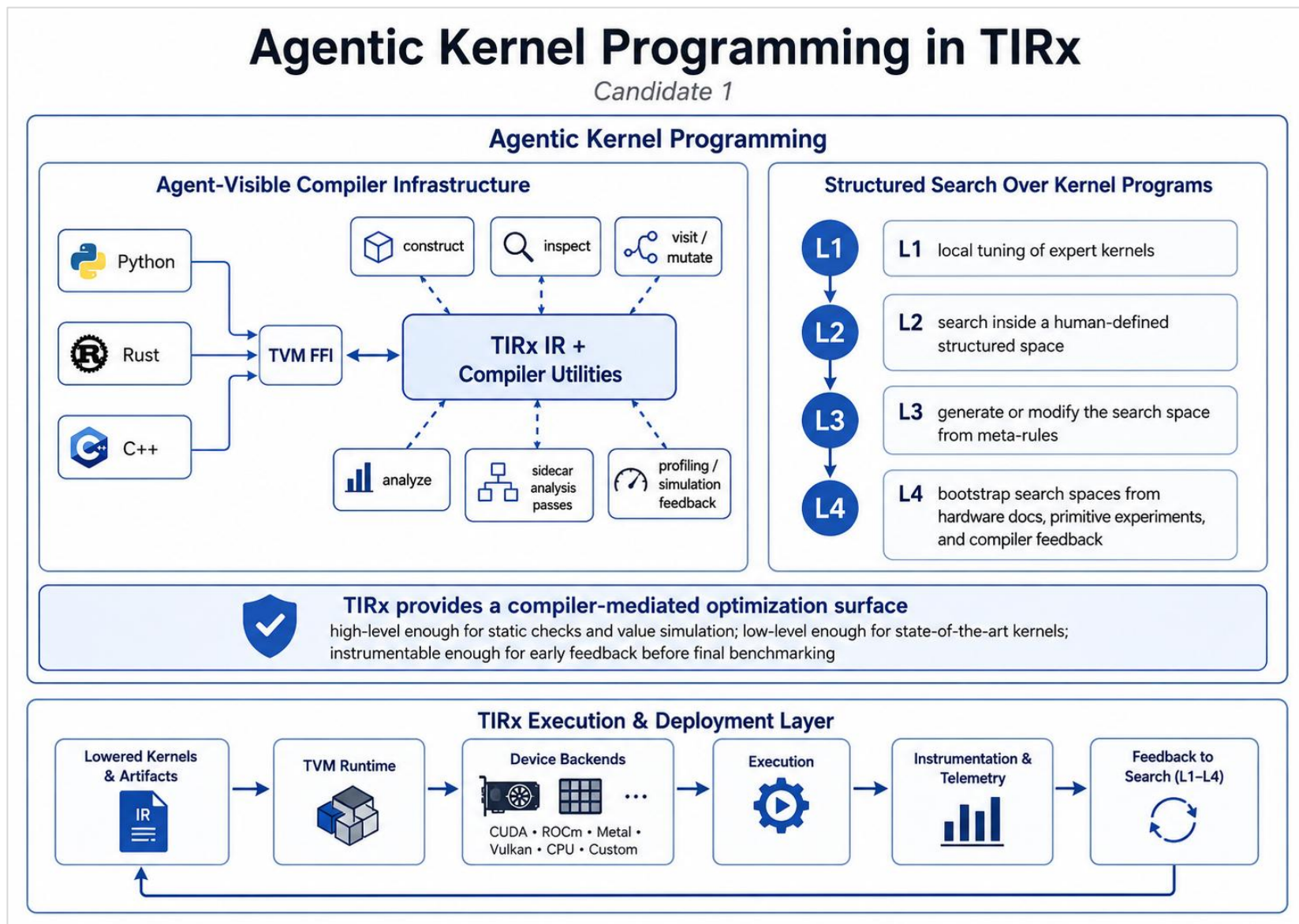
 调试阶段
 调优阶段

业界实践：TVM在Agentic编程上的探索

《Modern GPU Programming
For MLSys》

作者：TVM 陈天奇

High-level enough for static
checks and value simulation;
Low-level enough for state-
of-art kernel;
Instrumentable enough for
early feedback before final
benchmarking.



业界实践：FlyDSL的探索

1 仓库内置 Agent 脚手架

- **.claude/skills/** 18 个 git 追踪的技能
- **flydsl-kernel-authoring / debug-flydsl-kernel**
- **gemm-optimization / bisect-perf-regression**
- **CLAUDE.md** 专设「Agent Operating Guidelines」
- 文档地图 + 环境变量表 + kernel 路由指引
- 路由目标: “prevent likely agent mistakes”

3 Agent 可见的编译器与结构

- 显式 layout 代数 (Python DSL), 结构化可分析
- **tools/flydsl-lsp-server** IR 语言服务
- **fly-opt** 独立的 MLIR/IR pass 工具
- **COMPILE_ONLY** + 确定性 JIT 缓存, 缓存自动失效
- **L0/L1a/L1b/L2** 分层测试 + 本地风格门禁

2 编译前就给出密集反馈

- **DSLCompileError** 源码片段 + caret 定位
- 过滤框架栈帧, 只暴露用户级错误
- (tests/unit/test_diagnostics.py)
- **oob-detection** 静态区间分析查越界
- **FLYDSL_DUMP_IR / AST_DIFF** IR 可检视
- **print_typst.py** layout 渲染成可视网格

4 profiling → 分析闭环

- **capture-kernel-trace** rocprofv3 采 ATT trace
- **kernel-trace-analysis** 指令级热点分析
- **hotspot_analyzer.py** 读 VGPR/SGPR/LDS/occupancy
- **pmc_l2_analyzer.py** L2 cache 计数分析
- trace 目录名含 **ui_output_agent *_dispatch_***

算子生成 Agent Friendly DSL的动机

算子生成的要求

正确性

高质量实现算法

亲和性

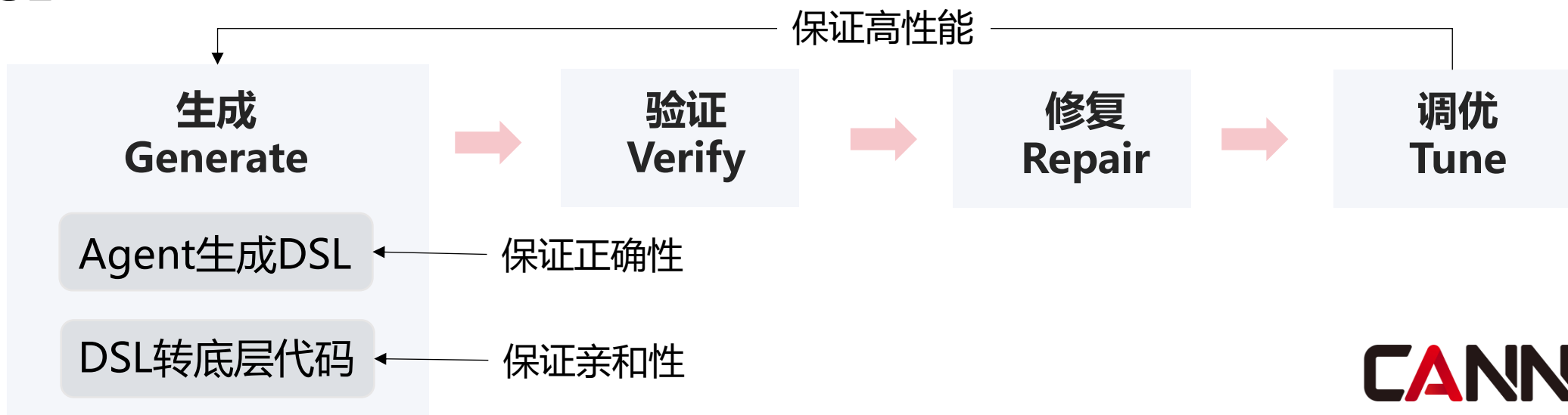
适配Asc语法和昇腾硬件

高性能

榨取芯片算力与带宽能力

三个目标，模型/Agent 难以一次搞定

引入DSL



算子生成 Agent Friendly 的原则

语义层

局部可推理

单一规范形式

API语义清晰

反馈层

结构化诊断

确定性反馈循环

算法与调度分离

可验证性优先

生态层

文档即代码、示例即类型

语料库性质

IR是一等公民

局部可推理不友好举例：

```
1091:  __aicore__ inline void Run() {  
      .....  
1097:  SET_FLAG(V, MTE2, EVENT_ID2);  
      .....  
1561:  ub_to_gm<ArchType::ASCEND_V220,...>()  
      .....  
1571:  SET_FLAG(MTE3, MTE2, pingpong_flag);  
      .....  
1818:  WAIT_FLAG(V, MTE2, EVENT_ID2);
```

相距 721 行

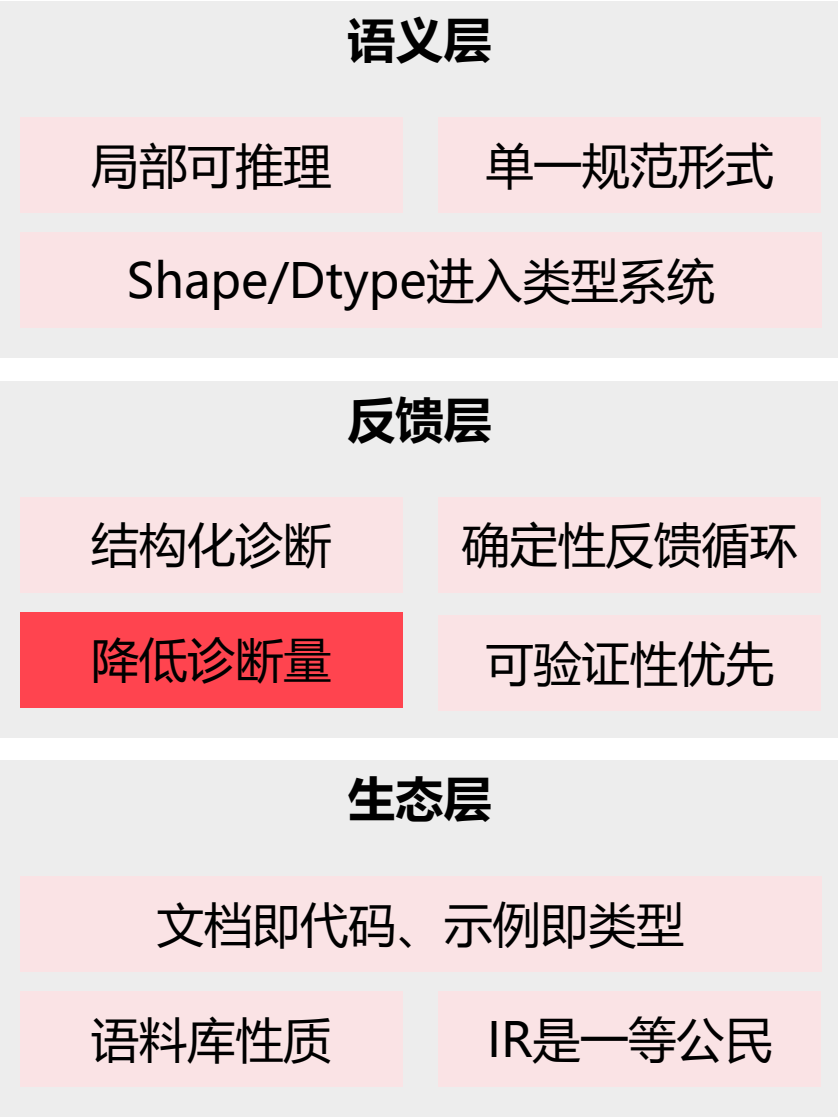
https://gitcode.com/cann/ops-transformer/blob/master/attention/incre_flash_attention/op_kernel/arch22/paged_attention_antiquantkv.h

```
352:  if (initKVG) {  
354:      WAIT_FLAG(MTE1, MTE2, PingFlag);  
      }  
      .....  
966:  if (initKEV) {  
967:      SET_FLAG(MTE1, MTE2, PingFlag);  
968:  }
```

相距 613 行

https://gitcode.com/cann/ops-transformer/blob/master/attention/incre_flash_attention/op_kernel/arch20/unpad_paged_attention_decoder.h

算子生成 Agent Friendly 的原则



手写代码:主路径 + 尾块路径两套代码,边界处理靠开发者保证

序列 S = 1000,CHUNK = 256 → 3 个整块 + 1 个尾块:

按 256 拷贝 → 越界 24



越界读不崩溃、不报错,只是把隔壁的数据当真值算 —— 静默精度错,无任何结构化诊断

主路径(整块,快路径)

```
CopyIn(k_l1, gm+off, 256*D);  
Compute(k_l1, 256);
```

尾块路径(手写慢路径)

```
tail = S - idx*256;           // 232  
CopyIn(k_l1, gm+off, tail*D); // 长度手算  
Pad(k_l1, tail, 256, 0);     // 手动补零  
SetMask(tail % VL);          // 手动掩尾拍  
ComputeTail(k_l1, tail);
```


探索点一：构建协程编程范式

问题所在

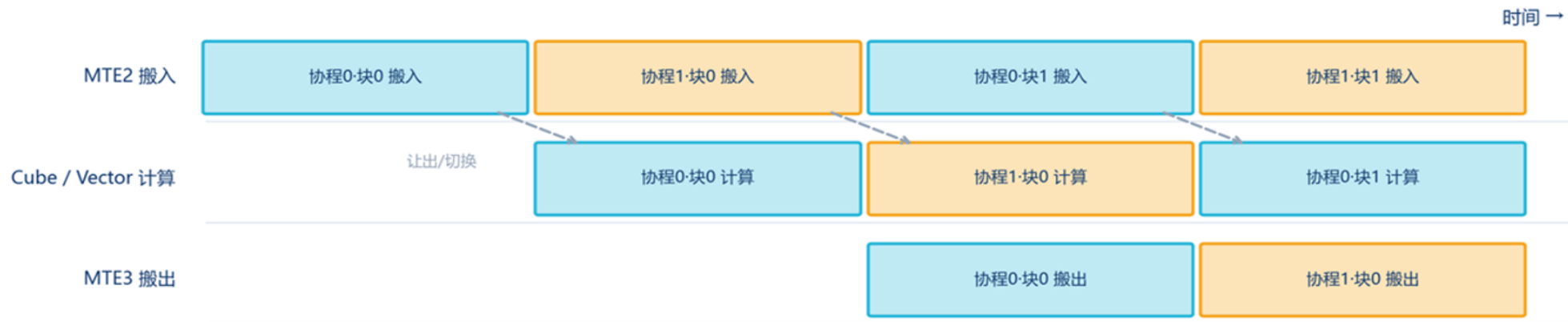
- N-Buffer (n=2为double buffer) 下不同n对应代码实现不同，不同策略下代码改动较大
- 不同n-buffer下要针对性的手动取 slot、算 (pp ^ = 1、buf[pp])

解法 • Buffer 定义时标 $depth=N$ ，其余交给框架

- DSL 在生成 Ascend C 时自动实现整套 N-buffer 流程
- 开发者用协程编程范式，只需改一个参数即可实现N-buffer

```
self.l0c_db = Channel(MemLoc.L0C,  
                      (CHUNK_SIZE, head_dim), float32, depth=2)
```

硬件执行视角:两个协程在多级流水上交替重叠,算力被填满



同一时刻:协程0 在算「块k」,协程1 在搬「块k+1」——搬运与计算重叠,PingPong 无需手写

协程在等待 Slot 就绪时自动让出执行权,框架在交接点插入精确的 SetFlag/WaitFlag

■ 协程 0(用 Slot 0)
■ 协程 1(用 Slot 1)

探索点二：消除开发者手插同步

问题所在

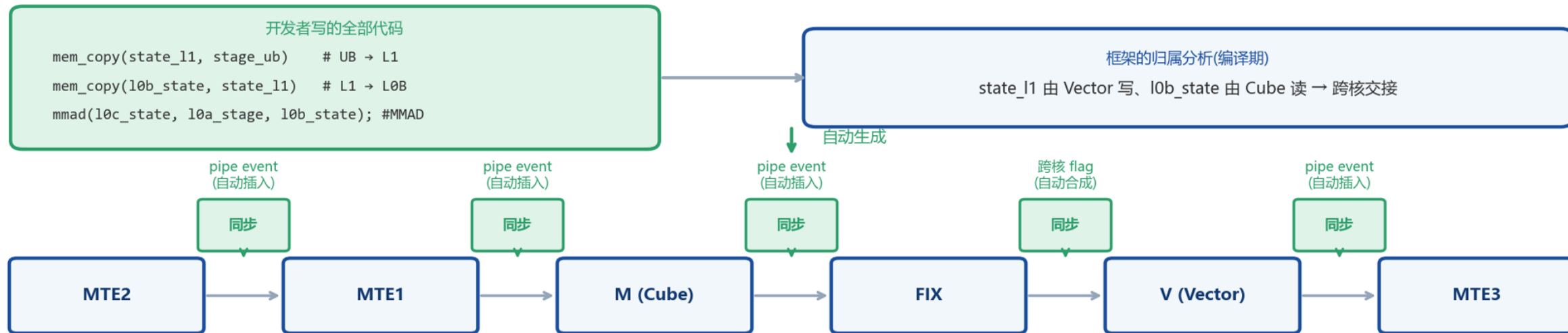
- Vector 与 Cube 并发，每个交接点都要手插同步
- 典型案例：
 - 一个复杂算子包含 50+ SetFlag/WaitFlag和 10+跨核 flag

解法 • 按 buffer 归属，自动补齐精确同步

- 声明谁写谁读，框架据归属合成生产-消费握手与跨核 flag
- 同步 bug 直接消失，且自动补出的就是最优同步

```
mem_copy(l0b_state, state_l1)
# 搬运 + 生产消费 + 跨核 flag 全由框架合成
```

CANNBot-DSL:开发者只写数据流,框架按「谁写谁读」自动补齐精确同步



探索点三：分离出性能调优因子

问题所在

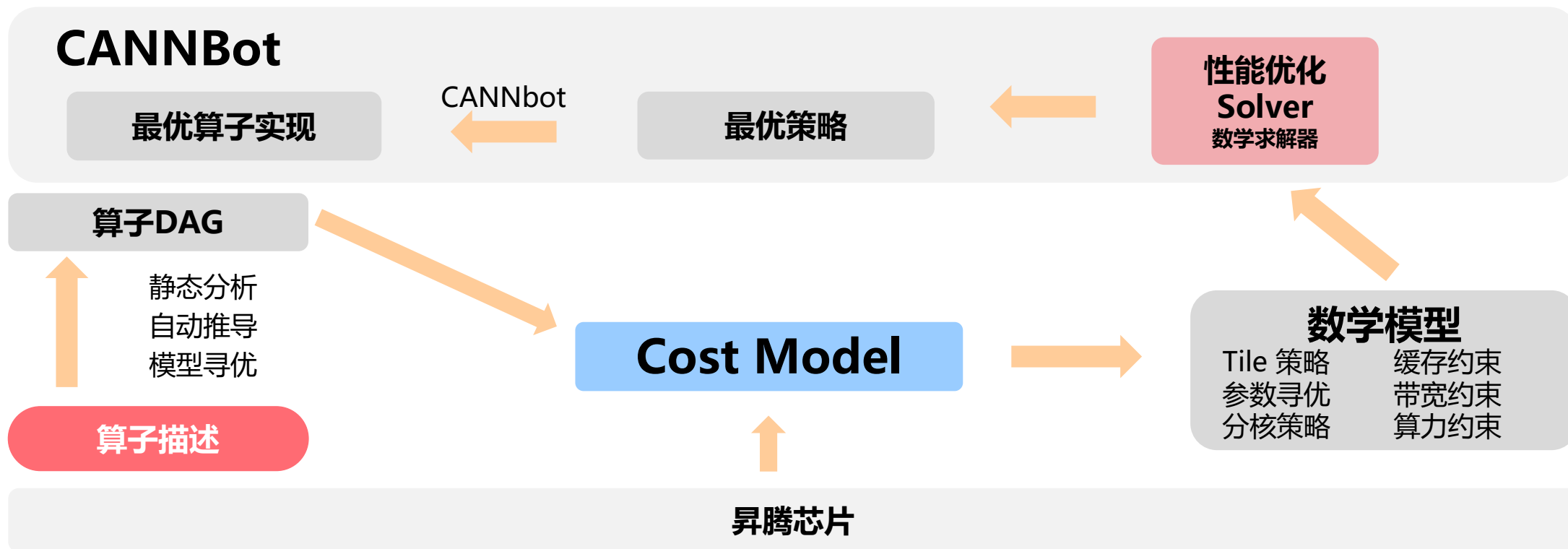
- 每轮只改一个点，但每个点落到手写代码都不是一个点
- 4 路展开改 8 路：循环体抄 8 份、手算偏移，mask / gamma 跟着抄
- 调双缓冲档位或每核 chunk 分组，会牵动 UB 预算 / 流水 / slot

解法 · 每个优化点收敛成一个参数

- 展开因子是循环上的一个参数：unroll=4 改 8 只动这个数
- 双缓冲加一档只改 depth=3；框架收走改写，判断仍归开发者

```
for base in dsl_range(0, dv_base*half_dk, VL, unroll=4)  
# 改 8 只动这一个数，其余不受牵连
```

性能
优化
求解器



更多探索方向

简化复杂的地址计算，声明式 buffer，偏移编译期推导

- 十几个中间量在片上手工摆放，地址靠 `off += 字节数` 一条链推
- 改一个量的大小或类型，后面所有偏移都得跟着挪

- `make_l1` 只给类型与形状，偏移 / 对齐 / 分形交框架
- `tile_view`、`local_slice` 零拷贝视图取子块，只是元数据

```
q_l1 = make_l1(BFloat16,  
              (CHUNK_SIZE, head_dim), data_format="nz")
```

隐藏硬件特定约束，格式与转置变显式参数，组合封装算子

- Cube 要求 L1 上是 NZ 分形，GM 是 ND，中间必须做 Nd2Nz
- 32B 对齐、round 模式、Fixpipe 手拼等隐性规矩，Agent 不擅长

- `make_copy_engine` 声明一次格式转换，转置退化成一个布尔
- `matmul` / `cast` / `exp` 等语义算子把隐性约束关进实现

```
mem_copy(mqk_l1, gm_tile, engine=self.gm2l1)  
mem_copy(l0b, u_l1, transpose=True)
```

简化尾块的处理，补齐 / 掩码 / 余量循环由框架生成

- 序列长度未必是 chunk 长整数倍，最后一块常是不满的尾块
- NZ 要 16 对齐，不对齐的边角要补齐

- `pad_value` 补边角、`update_mask` 掩尾拍、`dsl_range` 生成余量循环
- 主体只写一遍，尾块分支框架据形状统一生成，不会漏写

```
for base in dsl_range(0, dv_base*half_dk, VL):  
    vstore(ub, base, vmul(v, gamma, mask=mask))
```

编译期把该算的标量算掉

- 地址偏移、循环上下界、尾块大小等大量标量算术
- 留到运行时占标量流水 PIPE_S，卡在向量 / 矩阵计算前面

- 编译期常量经 @jit 的 self 带进 kernel，派生量在 lowering pass 一次算清
- 实测 scalar 降至约六成：AIC -36%、AIV -41% (12 组 shape 均值)

```
for base in dsl_range(0, dv_base*half_dk, VL)  
    # 步长 / 边界在编译期即为常量，NPU 上不再做标量算术
```

Agent Friendly生成高性能算子：把该收的收走，把该留的留下

DSL 负责 · 复杂实现和易错点

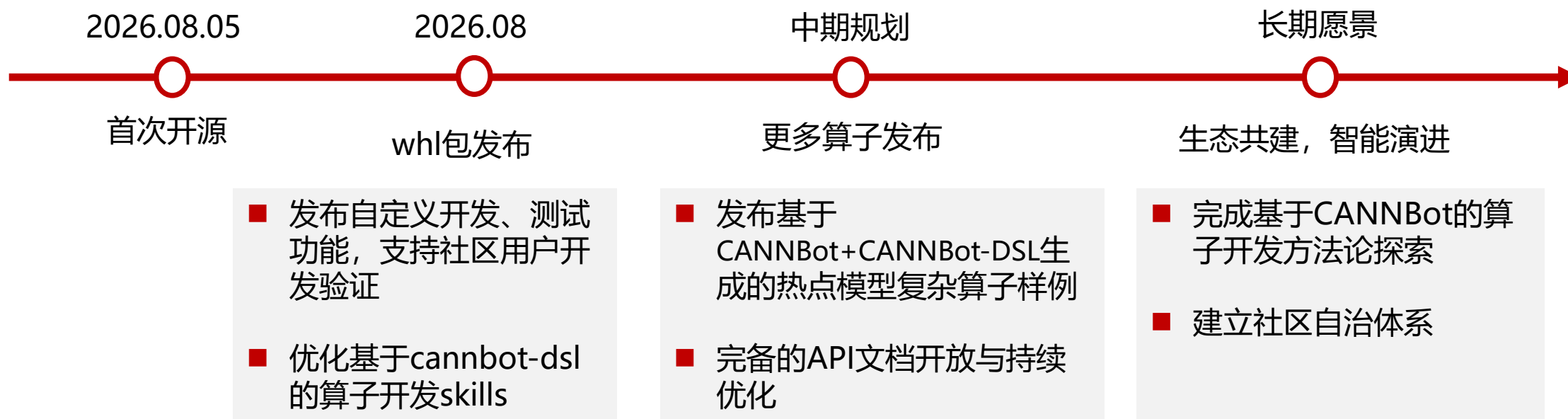
- 非局部依赖 · NBuffer轮换 · 复杂的同步插入
- API原子性不足 · 尾块补齐/掩码/余量循环多样
- 代码逻辑的大规模改动 · 调优时的机械改写

开发者负责 · 算法设计

- Tile 策略 · 分核策略 · N-Buffer策略
- 流水排布策略 · 数据预加载策略
- 数值格式选取 · 性能参数调优

算法还是那个算法——CANNBot-DSL只是把容易让Agent翻车、又不左右性能的硬件表达整个接过，让“写对”变成局部就能判断的事。

CANNBot-DSL Roadmap



代码仓地址: <https://gitcode.com/cann/cannbot-dsl>

欢迎前往仓库查阅体验!



CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

CANN